Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Qual è l'ordine dei bucket di origine e importanza, dal **meno** al **più** prioritario?

- A Autore, utente, user agent, autore `!important`, utente `!important`, user agent `!important`.
- B User agent, utente, autore, autore `!important`, utente `!important`, user agent `!important`.
- C User agent, autore, utente, utente `!important`, autore `!important`, user agent `!important`.
- D User agent `!important`, utente `!important`, autore `!important`, autore, utente, user agent.

Es. **2.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  const nome = "Ada";
2  const anni = 36;
3  console.log(`${nome} ha ${anni + 1} anni`);
4  console.log('${nome} ha ${anni + 1} anni');
```

- A `Ada ha 37 anni, Ada ha 361 anni.`
- B `Ada ha 37 anni, ${nome} ha ${anni + 1} anni.`
- C Viene sollevato un `SyntaxError`.
- D `Ada ha 361 anni, ${nome} ha ${anni + 1} anni.`

Es. **3.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  const voti = new Map([["web", 30], ["reti", 27]]);
2  voti.set("web", 28);
3  for (const [esame, voto] of voti) {
4    console.log(esame + ": " + voto);
5  }
```

- A `web: 30, reti: 27.`
- B `reti: 27, web: 28.`
- C `0: web,28, 1: reti,27.`
- D `web: 28, reti: 27.`

Es. **4.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
1  <ul><li>B</li></ul>
2  <script>
3    function li(testo) {
4      const e = document.createElement("li");
5      e.textContent = testo;
6      return e;
7    }
8    const ul = document.querySelector("ul");
9    ul.before("X");
10   ul.prepend("Z");
11   ul.after("Y");
12   console.log(ul.children.length, ul.childNodes.length);
13   console.log(ul.textContent);
14 </script>
```

- A 1 2, BZ.
- B 1 2, ZB.
- C 1 1, ZB.
- D 2 3, ZB.
- E 1 4, ZXBY.
- F 2 2, ZB.

SIMULAZIONE

Es. **5.** **(2 punti)** Nella sezione *Tempistiche* della scheda *Rete* dei DevTools, una richiesta ha una fase *In attesa* molto lunga. Che cosa indica?

- A Che la risposta era molto grande da scaricare.
- B Che la risoluzione DNS è stata lenta.
- C Che la richiesta è rimasta in coda nel browser perché non c'era connessione.
- D Che il browser ha impiegato molto tempo a disegnare la pagina.
- E Che la negoziazione TLS è fallita.
- F Che il server ha impiegato molto tempo a elaborare la richiesta prima di inviare il primo byte della risposta.

Es. **6.** **(2 punti)** Un server serve soltanto i file della sua *document root*. Quali delle seguenti esigenze **non** si possono soddisfare in questo modo?

- A Servire un foglio di stile CSS condiviso da tutte le pagine.
- B Servire la stessa pagina a tutti gli utenti, in HTTPS.
- C Aggiornare il contenuto della pagina nel browser con JavaScript.
- D Servire immagini e script JavaScript.
- E Registrare un ordine in seguito all'invio di un form.
- F Personalizzare una pagina per l'utente che la richiede.

Es. **7.** **(2 punti)** A che cosa serve `nodemon`?

- A A monitorare il traffico di rete del server.
- B A mostrare le variabili in scope durante il debug.
- C A scaricare le dipendenze mancanti all'avvio.
- D A installare e cambiare le versioni di Node.js.
- E A eseguire il server in più processi paralleli.
- F A riavviare automaticamente il server quando cambia il codice sorgente.

Es. **8.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono, nell'ordine, le richieste `GET /`, `GET /nonesiste`, `GET /todo` e `GET /todo` con l'header `Cookie: login=si`?

```
1  import express from "express";
2
3  const app = express();
4  const todoRouter = express.Router();
5
6  function richiediLogin(req, res, next) {
7    if (req.headers.cookie === "login=si") next();
8    else res.redirect("/auth");
9  }
10 todoRouter.get("/todo", richiediLogin, (req, res) => res.send("La lista"));
11 app.get("/", (req, res) => res.send("Home"));
12 app.use(todoRouter);
13 app.get("/*splat", (req, res) => res.status(404).send("Non trovato"));
14
15 app.listen(3000);
```

- A `200 Home, 404 Non trovato, 302 -> /auth, 200 La lista.`
- B `200 Home, 404 Non trovato, 302 -> /auth, 302 -> /auth.`
- C `200 Home, 404 Non trovato, 404 Non trovato, 200 La lista.`
- D `302 -> /auth, 302 -> /auth, 302 -> /auth, 200 La lista.`
- E `200 Home, 404 Non trovato, 200 La lista, 200 La lista.`
- F `200 Home, 302 -> /auth, 302 -> /auth, 200 La lista.`

SIMULAZIONE

Es. **9.** **(5 punti)** Si descrivano le differenze fra l'invio di un form con il metodo `GET` e con il metodo `POST`, indicando in quali casi conviene usare l'uno o l'altro.

Es. **10.** **(5 punti)** Si confrontino **Flexbox** e **Grid**: a che cosa servono, quali sono i concetti principali di ciascuno e quando conviene usare l'uno o l'altro.

Es. **11.** **(5 punti)** Un'applicazione memorizza lo stato nei cookie `auth` e `username`. Si descriva quali header si scambiano browser e server al login e nelle richieste successive, e si spieghi perché questo approccio non è sicuro.

SIMULAZIONE